Timing, Intensity, Energy, and Zero-Crossing Features

Detailed Explanation of Features 57–71

Technical Report

Abstract

This report documents a Python feature-extraction routine that calculates 15 recording-level timing, pause, amplitude, energy, and zero-crossing features from a mono speech waveform. It explains the code line by line, derives the underlying equations, motivates the 25 ms frame and 10 ms hop configuration, and discusses the limitations of simple RMS-threshold activity detection. The report also explains how these features may complement voice-quality and spectral features in Parkinson's disease speech-classification research.

Contents

1 Introduction

Speech contains information not only in its spectrum and voice quality but also in its temporal organization and amplitude dynamics. Timing features describe recording duration, active intervals, and pauses. Amplitude-derived features summarize the level and variation of the waveform. Short-time energy measures squared amplitude, while zero-crossing rate (ZCR) describes how frequently the waveform changes sign.

The implementation discussed here returns 15 scalar features numbered 57–71. These feature numbers are identifiers in a larger feature inventory.

2 Input preparation and validation

The function begins with:

```
y = np.asarray(y, dtype=np.float32).squeeze()
if y.ndim != 1 or len(y) == 0:
    raise ValueError("y must be a non-empty, mono waveform.")
if not np.all(np.isfinite(y)):
    raise ValueError("Waveform contains NaN or infinite values.")
if sr <= 0:
    raise ValueError("Sampling rate must be positive.")
```

`np.asarray` converts the waveform to a 32-bit floating-point NumPy array. `squeeze` removes dimensions of size one, such as a singleton channel dimension. The dimensionality check requires a one-dimensional mono waveform and rejects empty input. `np.isfinite` checks that every sample is neither NaN nor infinite. Finally, the sampling rate must be positive. A robust production implementation should also check that the sampling rate is finite and appropriate for the signal.

These checks are important because invalid arrays can lead to meaningless frame calculations or failures inside librosa.

3 Feature 57: recording duration

The code uses

$$T_{\text{recording}} = \frac{N}{f_s},$$

where N is the number of samples and f_s is the sampling rate in samples per second. For example, 160000 samples at 16 kHz represent 10 seconds. Duration is a basic temporal descriptor, but it depends on the recording task and amount of speech. It is not by itself a speaking-rate measure.

4 Frame-based analysis

The implementation defines

```
frame_length = int(0.025 * sr)
hop_length = int(0.010 * sr)
```

The nominal frame duration is 25 ms and the hop duration is 10 ms. At 16 kHz, these correspond to 400 samples and 160 samples, respectively. Frames overlap by 240 samples, or 15 ms. Short-time analysis is used because speech changes over time; a single recording-level amplitude value would hide local changes.

The function rejects recordings shorter than one complete frame. It then computes frame-level RMS:

```
rms = librosa.feature.rms(
    y=y,
    frame_length=frame_length,
    hop_length=hop_length,
    center=False
)[0]
```

For frame i with L samples,

$$R_i = \sqrt{\frac{1}{L}\sum_{n=0}^{L-1} x_i[n]^2}.$$

Squaring makes positive and negative samples contribute positively; averaging summarizes the frame; taking the square root returns the result to amplitude units. RMS is useful for tracking changes in signal level over time.

`center=False` means librosa does not pad the waveform to center frames at the boundaries. The `[0]` selects the single RMS sequence from the returned feature array.

5 Frame timestamps

The code calculates

$$t_i = \frac{iH + L/2}{f_s},$$

where H is the hop length and L is the frame length. This is the center time of each frame. The resulting `frame_times` array is not used later in the current function, but it can be used to plot the RMS contour or align frame-level activity estimates with time.

6 Energy-based activity detection

The code finds the maximum RMS value and defines a threshold:

$$\theta = \max(0.01R_{\max}, 10^{-6}).$$

A frame is marked active when

$$\text{active}_i = (R_i > \theta).$$

The implementation calls this Boolean array `voiced`, but that name is not physiologically precise. RMS thresholding detects high-energy frames, not vocal-fold voicing. A high-energy unvoiced fricative or background sound may be marked active, while quiet voiced speech may be marked inactive. A more accurate variable name is `active_frames` or `speech_activity_estimate`.

The relative threshold of 1 percent of the maximum RMS is a heuristic. It can be affected by loud transients, background noise, and recording gain. A proper voice-activity detector or annotated speech boundaries are preferable when accurate speech and pause timing are required.

7 Feature 58: estimated speech duration

The code calculates

$$T_{\text{active}} = N_{\text{active}}\frac{H}{f_s},$$

where N_{active} is the number of frames marked active. The hop duration is used rather than the full frame duration because adjacent frames overlap. Counting every active frame as a full 25 ms interval would repeatedly count overlapping audio.

This is an approximation: the first and last frame boundaries and transitions are represented on the hop grid. It is an energy-based active duration, not a manually verified speech duration.

8 Feature 59: estimated silence duration

The code computes

$$T_{\text{inactive}} = \max(0, T_{\text{recording}} - T_{\text{active}}).$$

This is the portion of the recording not counted as active. It may include true silence, pauses, quiet speech, or frames below the energy threshold. The label `silence_duration` should therefore be interpreted as estimated inactive duration under the chosen detector.

9 Detecting inactive runs

The Boolean array is inverted:

```
unvoiced = ~voiced
```

This produces a mask of frames classified as inactive. The code pads the sequence with `False` at both ends, converts it to integers, and applies a first difference:

```
changes = np.diff(
    np.concatenate(([False], unvoiced, [False])).astype(int)
)
starts = np.where(changes == 1)[0]
ends = np.where(changes == -1)[0]
```

A transition from 0 to 1 marks the beginning of an inactive run; a transition from 1 to 0 marks its end. Padding ensures runs touching the first or last frame are also detected. The run length in frames is `ends - starts`.

The estimated duration of each run is

$$P_i = (e_i - s_i)\frac{H}{f_s}.$$

The code retains only runs at least 0.10 seconds long. This removes short energy dips that might otherwise be counted as pauses. The 100 ms threshold is a heuristic and should be selected according to the speech task and the study's pause definition.

10 Features 60–63: pause statistics

After filtering, the number of pauses is the length of the pause-duration array:

$$K = \text{len}(P).$$

The four pause features are:

$$\text{Number of pauses} = K,$$

$$\text{Mean pause duration} = \frac{1}{K}\sum_{i=1}^{K} P_i,$$

$$\text{Pause-duration standard deviation} = \sqrt{\frac{1}{K}\sum_{i=1}^{K}(P_i - \overline{P})^2},$$

$$\text{Maximum pause duration} = \max_i P_i.$$

If no pause passes the threshold, the mean, standard deviation, and maximum are set to NaN because they are undefined for an empty set. NumPy's `std` uses `ddof=0` by default, corresponding to the population-style standard deviation.

These features describe the inactive intervals detected by the algorithm. They should not be described as exact linguistic pauses unless the detector has been validated against speech annotations.

11 Feature 64: speech-to-total-duration ratio

The code calculates

$$R_{\text{speech}} = \frac{T_{\text{active}}}{T_{\text{recording}}},$$

provided recording duration is positive. The ratio is dimensionless and lies approximately between zero and one when the active-duration estimate is well behaved. It describes the fraction of the recording classified as active. It is not speaking rate, which would require a measure such as syllables or words per unit time.

12 Features 65–68: intensity-like statistics

The implementation converts RMS amplitude to a logarithmic scale:

$$L_i = 20 \log_{10}(\max(R_i, 10^{-10})).$$

The floor prevents the logarithm of zero. The factor 20 is used for amplitude ratios because power is proportional to amplitude squared. The code then calculates the mean, standard deviation, minimum, and maximum of L_i.

These are digital RMS levels relative to an implicit amplitude reference of one in the waveform's numerical units. They are not calibrated sound-pressure levels in dB SPL. If waveform gain changes, these values change. Microphone calibration and a known reference are required for absolute acoustic-level measurements.

The four outputs are:

- Mean intensity-like level: average frame-level logarithmic RMS amplitude.
- Standard deviation: variation in frame-level logarithmic amplitude.
- Minimum: quietest frame according to the digital RMS calculation.
- Maximum: loudest frame according to the digital RMS calculation.

The mean of logarithmic values is not the same as taking the logarithm of mean RMS. The two operations should not be confused.

13 Features 69–70: short-time energy

The code defines

$$E_i = R_i^2 = \frac{1}{L} \sum_{n=0}^{L-1} x_i[n]^2.$$

This is mean-square amplitude per frame. It is sometimes called short-time energy in practical feature pipelines, although unnormalized frame energy would use the sum rather than the mean. The function calculates the mean and standard deviation of E_i across frames.

Mean energy summarizes average squared signal magnitude. Energy standard deviation describes how much frame-level squared amplitude varies. These measures are related to the RMS-derived decibel features because all are computed from the same RMS sequence. They are not independent physical observations.

14 Feature 71: zero-crossing rate

A zero crossing occurs when adjacent waveform samples change sign. A common normalized expression is

$$Z_i = \frac{1}{2L} \sum_{n=1}^{L-1} \left| \operatorname{sgn}(x_i[n]) - \operatorname{sgn}(x_i[n-1]) \right|.$$

The exact treatment of zero-valued samples and boundary normalization follows the library implementation. Librosa calculates a frame-wise ZCR sequence using the same frame length, hop length, and `center=False` setting. The function returns its mean:

$$\overline{Z} = \frac{1}{N_f} \sum_{i=1}^{N_f} Z_i.$$

ZCR can reflect aspects of spectral content and noise-like behavior. It is not a direct pitch estimate or a direct measure of vocal-fold vibration.

15 Complete feature inventory

No.	Feature key	Meaning
57	`recording_duration`	Total waveform duration in seconds.
58	`estimated_speech_duration`	Duration assigned to active frames by RMS thresholding.
59	`silence_duration`	Total duration minus estimated active duration.
60	`number_of_pauses`	Number of inactive runs lasting at least 100 ms.
61	`mean_pause_duration`	Mean duration of qualifying inactive runs.
62	`std_pause_duration`	Population-style standard deviation of pause durations.
63	`max_pause_duration`	Longest qualifying inactive run.
64	`speech_to_total_duration_ratio`	Estimated active duration divided by total duration.
65	`mean_intensity`	Mean logarithmic RMS amplitude.
66	`std_intensity`	Standard deviation of logarithmic RMS amplitude.
67	`min_intensity`	Minimum logarithmic RMS amplitude.
68	`max_intensity`	Maximum logarithmic RMS amplitude.
69	`mean_short_time_energy`	Mean frame-level mean-square amplitude.
70	`std_short_time_energy`	Standard deviation of frame-level mean-square amplitude.
71	`zero_crossing_rate`	Mean normalized frame-level sign-change rate.

16 Why these features may be useful in PD speech research

Parkinson's disease can be associated with changes in speech motor control, phonation, articulation, and prosody. Timing and pause features may describe temporal organization; amplitude statistics may describe signal-level dynamics; and ZCR may add a coarse descriptor of waveform behavior. These features can complement voice-quality features such as jitter, shimmer, HNR, CPP, and spectral features such as MFCCs.

They are not disease-specific. Recording duration depends on task design; pauses depend on language, reading ability, and task difficulty; intensity depends on microphone gain and distance; and ZCR depends on phonetic content and noise. Interpretation therefore requires careful control of acquisition and task conditions.

17 Implementation limitations and recommended improvements

1. Rename `voiced` to `active_frames` because the detector is based on RMS energy, not pitch or vocal-fold periodicity.
2. If precise speech/pause segmentation is required, use a validated voice-activity detector or manual/automatic speech boundaries validated against annotations.
3. Reconsider the relative threshold based on recording conditions. A loud transient can raise the maximum RMS and change the threshold for the entire recording.
4. The 100 ms pause threshold is a heuristic. Report it and test sensitivity to reasonable alternatives.
5. The `frame_times` variable is calculated but unused; it can support visualization and debugging.
6. `silence_duration` is an estimate of inactive time and should not be interpreted as exact silence.
7. The intensity features are not dB SPL without calibration. Use consistent recording gain and microphone setup.
8. The short-time energy variable is mean-square amplitude, not unnormalized total frame energy.
9. Handle NaN values explicitly. Missing pause statistics should not be silently replaced with zero.
10. If multiple recordings belong to the same participant, split by participant where feasible to avoid identity leakage.

18 Conclusion

The function extracts 15 recording-level descriptors from timing, energy, amplitude, and waveform sign changes. Its central mechanism is short-time RMS analysis using 25 ms frames and 10 ms hops. The active/inactive mask is a simple energy-threshold approximation, so speech duration and pause statistics are estimates rather than validated linguistic boundaries. The amplitude-derived decibel values are not calibrated SPL, and the energy values are mean-square amplitude. With these qualifications documented and consistent preprocessing applied, the features can be evaluated as complementary inputs to a PD speech-classification model.

References

[1] B. McFee et al., "librosa: Audio and music signal analysis in Python," documentation and software, https://librosa.org/.

[2] L. R. Rabiner and R. W. Schafer, *Theory and Applications of Digital Speech Processing*. Pearson, 2011.